GMI German-Malaysian Institute

DIPLOMA PROGRAMME

MINI PROJECT

Academic Period : JANUARY 2026

Program	:	**DIPLOMA IN SOFTWARE ENGINEERING**
Course	:	**IOT APPLICATION & SOFTWARE**
Code	:	**SDT 2133**
Duration	:	**3 WEEKS**
Total Marks	:	**40**
Title	:	**MOTIONLIVE**

Student's Name :

(Individual / Group)

4. NUR FATIN FASISHAH BINTI M.RUSDI@RUSDEI

TTO's Name :

Semester/Trade/Group : _____

Date : _____

Mini Project Rubric Scale

Criteria	Needs Improvement (1 pt)	Fair (2 pts)	Good (3 pts)	Excellent (4 pts)
Hardware Setup	Incomplete or incorrect wiring; components not functioning	Some components have issues or missing parts	Minor wiring issues; all components functional	Correct and neat wiring; components work flawlessly
Microcontroller / Single Board Computer (SBC)	The setup is incomplete or not functioning	Limited functions used; code may have minor issues	Used correctly; basic features (Wi-Fi, GPIO) implemented	Code is well-integrated; utilizes both GPIO and communication features
Sensor/ Actuator Integration	No sensor/actuator or not functioning properly	Sensor/actuator integration is attempted with issues	At least one sensor/actuator integrated correctly	Multiple sensors/actuators integrated successfully
Programming Skills	Code doesn't run or is poorly structured	Code runs but is messy or hard to understand	Code is functional and readable; minor logic flaws	Efficient and well-documented code; logic is clear and modular
Communication Protocols	No communication protocol used or fails	Partial or unstable communication	Communication works but with occasional issues	Proper use of MQTT/HTTP; stable cloud connection
Cloud / Data Handling	No cloud connection or data handling present	Attempted cloud integration, but limited success	Data reaches cloud with minor delay or bugs	Data is reliably sent to and visualized on cloud platform
UI / Dashboard (Web/App)	No UI or not working	UI has limited functionality or bugs	UI is functional but basic or unpolished	Functional and user-friendly UI (mobile/web) with data/control

Creativity & Usefulness	Project lacks clear purpose or innovation	Project has basic functionality but lacks real-world use	Project shows creativity and purpose	Project is innovative, well thought-out, and solves a real problem
Documentation	No documentation or unclear	Minimal documentation or missing steps	Mostly clear with some missing details	Clear explanation, diagrams, and steps provided
Overall Functionality	Non-functional or incomplete	Partially functional	Mostly functional with minor bugs	Fully functional, stable, and meets goals

Mini Project Assignment Brief

🧠 Objective:

Design and build a functional IoT system using the *Microcontroller / SBC* that senses environmental data and communicates it over the internet. Your project must **collect** data, **process** it, and **display** it via a cloud platform, web interface, or mobile app.

📍 Learning Outcomes:

By the end of this project, you should be able to:

1. Understand and set up a microcontroller / SBC.
2. Interface sensors/actuators with microcontroller / SBC.
3. Program microcontroller / SBC to collect and send data.
4. Use communication protocols such as Wi-Fi and MQTT/HTTP.
5. Display data via a dashboard (web, mobile, or cloud).
6. Document and present your IoT solution clearly.

🧩 Project Requirements:

You are required to complete an IoT project that includes the following components:

Component	**Description**
Microcontroller / SBC	*Hibiscus Sense ESP32 Development Board / Raspberry Pi Kits*
Sensors/Actuators	At least 1 sensor (*e.g., DHT11*) and optionally 1 actuator (*e.g., LED, motor*)
Connectivity	Wi-Fi required; MQTT/HTTP optional
Cloud/Dashboard	Use a platform like *Blynk, Thingspeak, Firebase*, or a custom dashboard
UI (Optional)	Simple web page or mobile app for visualization or control

Documentation	Report including schematic, code, and explanation

📦 Deliverables:

You must deliver the following:

1. Functional IoT Prototype (live test demo)

2. Source Code (well-commented)

3. Circuit Diagram (manually drawn or using *Tinkercad/Fritzing*)

4. Project Report (in PDF, includes explanation, screenshots, issues faced)

5. Presentation (Slides to showcase your project in class)

🧱 Report Contents:

1. Introduction
2. Problem Statements
3. Objectives
4. Features
5. Hardware Specification – *Board, Sensors, Actuators*
6. Software Specification – *Cloud/Dashboard, UI*
7. Diagram
8. Screenshots(?)
9. Coding
10. Issues – *What difficulties have occurred*
11. Conclusion – *What I have learned, Future Recommendations*
12. References

Final Submission: **26 May 2026**

TABLE OF CONTENTS

1.0 Introduction **7**
2.0 Problem Statements **8**
3.0 Objectives **9**
4.0 Features **10**
4.1 Motion Detection 10
4.2 LED Alert Indicator 10
4.3 Buzzer Alarm System 10
4.4 Real-Time Cloud Monitoring 10
4.5 Automated System Operation 10
4.6 Data Visualization Dashboard 10
4.7 Internet Connectivity 11
5.0 Hardware Specification – Board, Sensors, Actuators **12**
6.0 Software Specification – Cloud/Dashboard, UI **15**
7.0 Diagram **16**
8.0 Screenshots **17**
9.0 Coding **21**
10.0 Issues – What difficulties have occurred **23**
10.1 PIR Sensor Sensitivity 23
10.2 Buzzer Connection Problem 23
10.3 Difficulty Connecting to ThingSpeak 23
10.4 Circuit Diagram and Diode Confusion 23
10.5 Difficulty Finding Raspberry Pi Simulator 23
11.0 Conclusion – What I have learned, Future Recommendations **24**
12.0 References **25**

1.0 Introduction

The Internet of Things (IoT) is a technology that allows devices and sensors to communicate and exchange data through the internet. IoT systems are widely used in many industries such as healthcare, smart homes, agriculture, and security systems. By using sensors and cloud platforms, IoT systems can monitor activities in real time and improve efficiency, safety, and convenience.

This project focuses on developing an IoT-based security monitoring system using Raspberry Pi 4, PIR motion sensor, LED, buzzer, and ThingSpeak cloud platform. The PIR sensor is used to detect human movement, while the LED and buzzer act as warning indicators when motion is detected. At the same time, the Raspberry Pi sends the sensor data to the ThingSpeak dashboard for online monitoring.

The purpose of this project is to create a simple and effective security system that can monitor movement in a specific area. The system can help users detect intruders and provide real-time alerts through both hardware components and cloud monitoring. This project also helps students understand the practical implementation of IoT technology, hardware integration, cloud communication, and Python programming.

2.0 Problem Statements

Nowadays, security and monitoring systems are very important in homes, offices, and restricted areas. Traditional security systems are usually expensive and difficult to monitor remotely. Some systems also lack real-time notification and cloud monitoring features.

In addition, many people need a low-cost and simple security solution that can detect suspicious movement automatically. Without an automated monitoring system, users may not be aware of unauthorized access or intruder activities in their area.

Another problem is that manual monitoring requires continuous human attention, which is inefficient and time-consuming. Therefore, an IoT-based security system is needed to provide automatic motion detection, alert notification, and real-time data monitoring through the internet.

This project aims to solve these problems by developing a motion detection security system using Raspberry Pi and ThingSpeak cloud platform.

3.0 Objectives

The main objectives of this project are to design and develop an IoT-based security monitoring system that is capable of detecting human movement and providing real-time monitoring through cloud technology. The objectives of this project are as follows:

1. To develop an IoT-based security monitoring system using Raspberry Pi 4.
2. To detect human movement using a PIR motion sensor.
3. To activate LED and buzzer alerts when motion is detected.
4. To send motion detection data to the ThingSpeak cloud platform.
5. To display real-time monitoring data on an online dashboard.
6. To improve knowledge and skills in IoT development, Python programming, and hardware integration.

4.0 Features

The IoT Security Monitoring System includes several important features that improve security monitoring and real-time alert management.

4.1 Motion Detection

The system uses a PIR (Passive Infrared) sensor to detect human movement within the monitoring area. When movement is detected, the sensor sends a signal to the Raspberry Pi for further processing.

4.2 LED Alert Indicator

A red LED is used as a visual alert indicator. The LED will automatically turn on whenever motion is detected, allowing users to easily identify that an intrusion or movement has occurred.

4.3 Buzzer Alarm System

The buzzer acts as an audio alert system. When the PIR sensor detects movement, the buzzer produces a warning sound to alert nearby users about possible intruder activity.

4.4 Real-Time Cloud Monitoring

The system sends motion detection data to the ThingSpeak cloud platform through an internet connection. Users can monitor the sensor activity and alert status remotely using the online dashboard.

4.5 Automated System Operation

The entire system operates automatically without requiring manual control. The Raspberry Pi continuously monitors the PIR sensor and activates alerts whenever movement is detected.

4.6 Data Visualization Dashboard

The ThingSpeak dashboard displays motion activity data in graphical form, making it easier for users to analyze and monitor intrusion activity in real time.

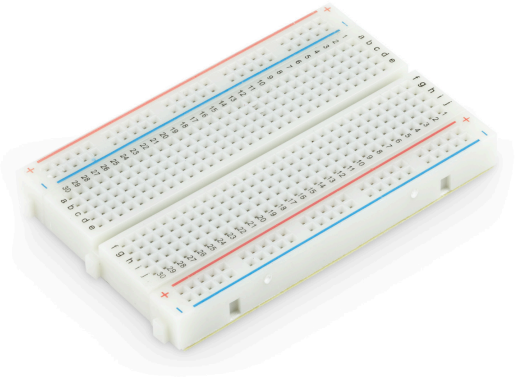
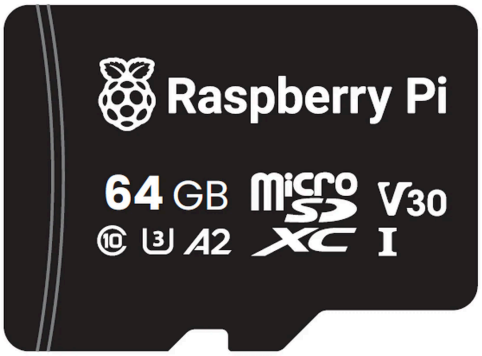
4.7 Internet Connectivity

The Raspberry Pi uses Wi-Fi connectivity to communicate with the cloud platform, enabling remote monitoring and data transfer through IoT technology.

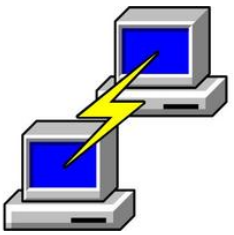
5.0 Hardware Specification – *Board, Sensors, Actuators*

1.	220-ohm Resistor	
2.	LED Red	
3.	Buzzer	

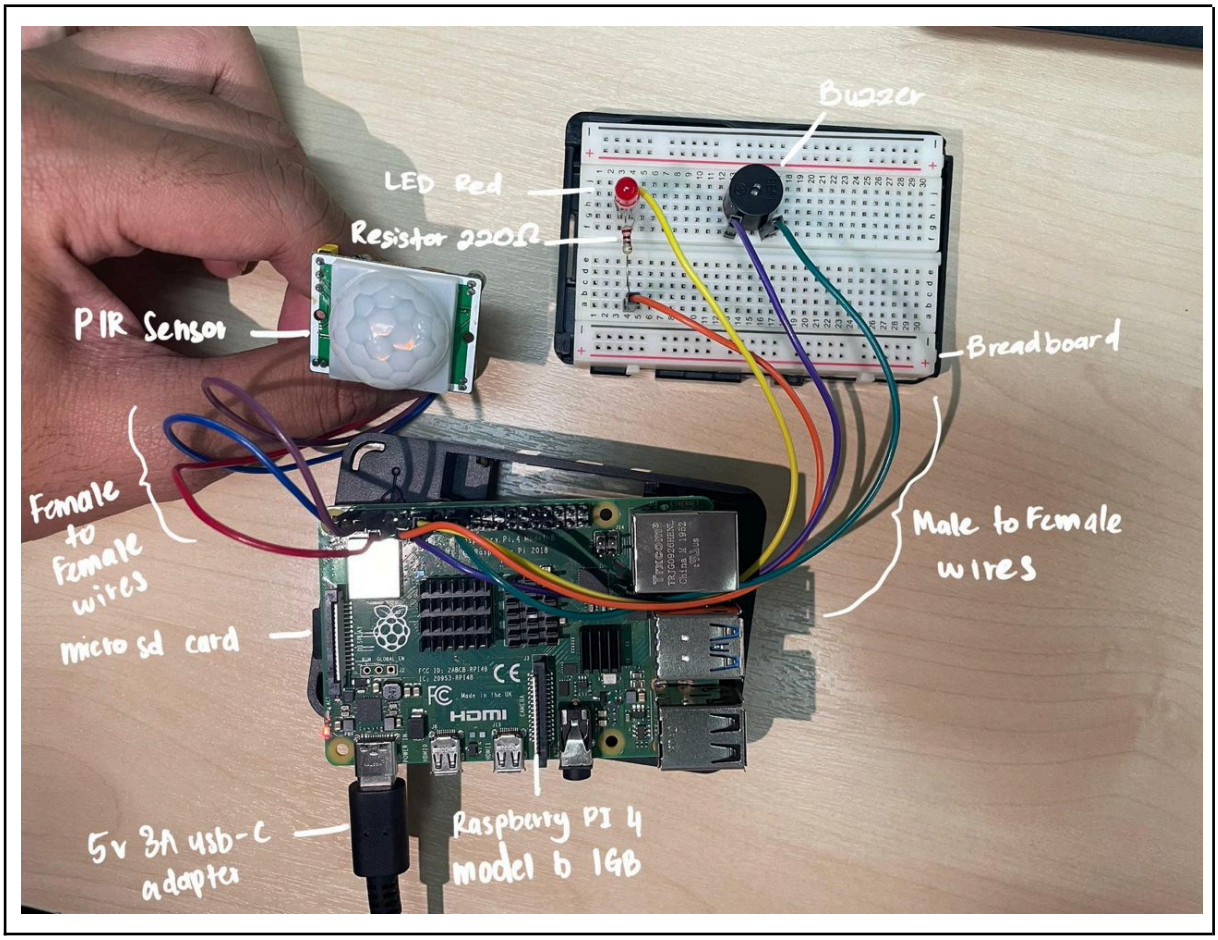
4.	Male to Female Wires	
5.	Female to Female Wires	
6.	PIR Sensors	

7.	Raspberry PI 4 Model B 1GB	
8.	Breadboard	
9.	5V 3A USB - C Adapter	
10.	Micro SD Card	

6.0 Software Specification – *Cloud/Dashboard, UI*

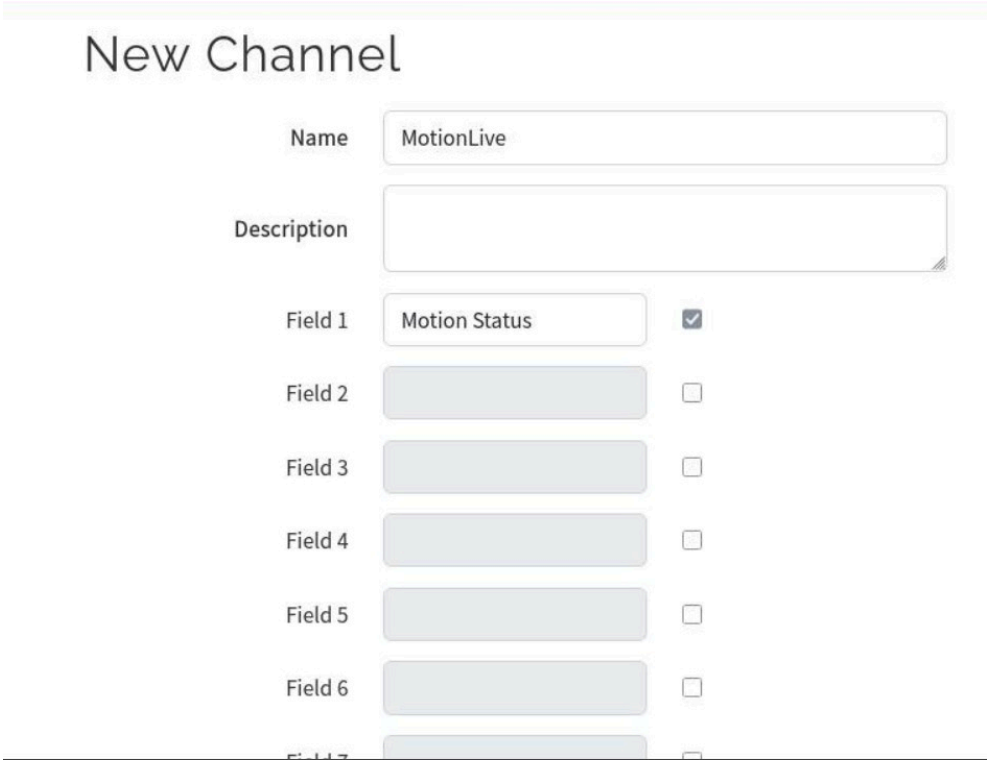
1.	Tiger VNC	
2.	PuTTY	
3.	ThingSpeak	
4.	Raspberry PI 4	
5.	Python	

7.0 Diagram

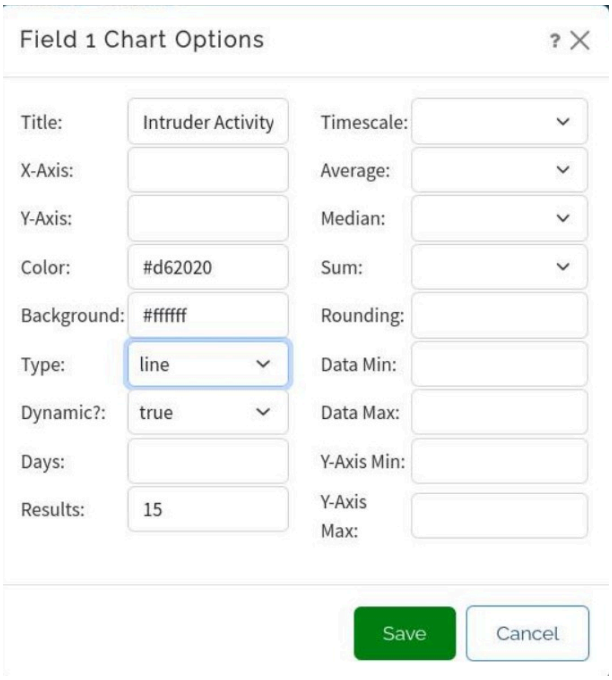

Buzzer
LED Red
Resistor 220Ω
PIR Sensor
Breadboard
Female to Female wires
Male to Female wires
micro sd card
5v 3A usb-C adapter
Raspberry PI 4 model b 1GB

8.0 Screenshots

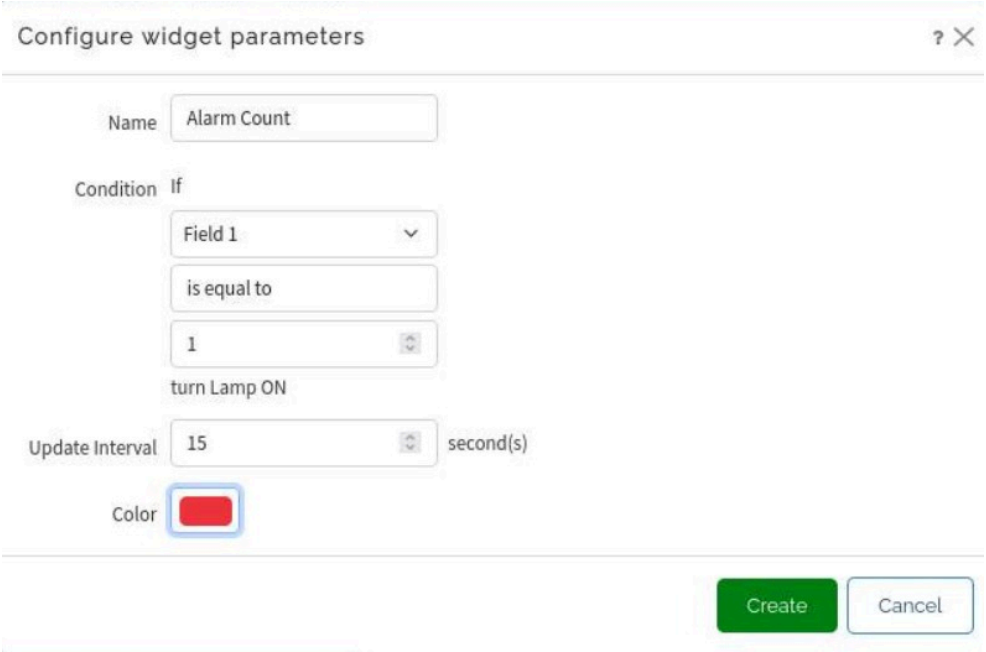
1. Create New Channel on ThinkSpeak. Insert MotionLive on Name textbox and Motion Status on Field 1 textbox. Lastly, click Save Channel.



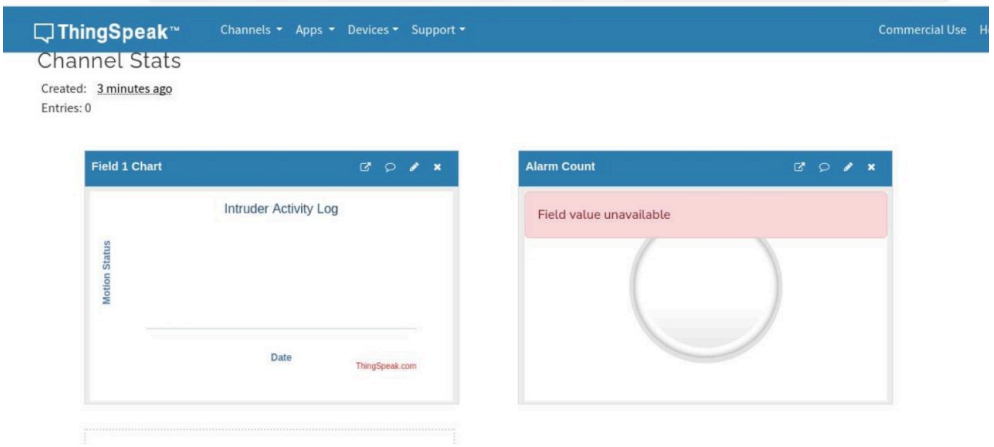
2. Fill the Field 1 Chart Options and click Save.



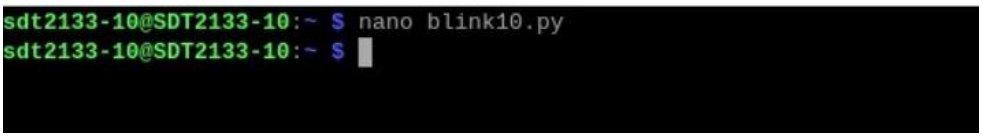
3. Create Lamp Indicator widget and edit the widget parameters. Then, click Create to save.



4. This dashboard shows the initial results on dashboards.



5. Go to terminal commands, insert nano blink10.py. Insert python code.



6. Python code

```
 GNU nano 8.4                                                    blink10.py
import RPi.GPIO as GPIO
import time
import requests  # Handles the HTTP connection to ThingSpeak

# Use BCM numbering for Raspberry Pi GPIO pins
GPIO.setmode(GPIO.BCM)

# Pin Setup
PIR_PIN = 4
LED_PIN = 17
BUZZER_PIN = 27

# ------------------------------------------------
# THINGSPEAK CLOUD CONFIGURATION
# ------------------------------------------------
# ⚠ Make sure to paste your actual Write API Key between the quotes!
API_KEY = "756GZGCM5LH7SXMS"
URL = "https://api.thingspeak.com/update"

# Configure hardware pin directions
GPIO.setup(PIR_PIN, GPIO.IN)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# Force actuators off initially to guarantee a clean state
GPIO.output(LED_PIN, False)
GPIO.output(BUZZER_PIN, False)

print("================================================")
print("     SDT 2133 IOT SECURITY SYSTEM INITIALIZING    ")
print("================================================")
print("Warming up PIR sensor... Please wait 20 seconds.")
time.sleep(20)  # Crucial calibration window to prevent false startup alarms
print("System Live! Monitoring for intruder movements...\n")

try:
    while True:

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo
```

7. Run the code by insert python3 blink10.py commands.

```
sdt2133-10@SDT2133-10:~ $ python3 blink10.py
```

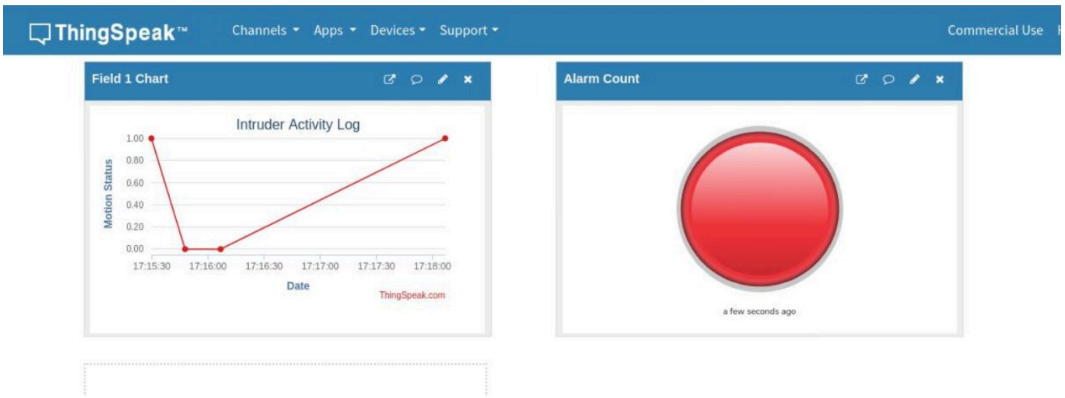
8. The results when system is running.

```
sdt2133-10@SDT2133-10:~ $ nano blink10.py
sdt2133-10@SDT2133-10:~ $ python3 blink10.py
================================================
     SDT 2133 IOT SECURITY SYSTEM INITIALIZING
================================================
Warming up PIR sensor... Please wait 20 seconds.
System Live! Monitoring for intruder movements...

INTRUDER DETECTED! Triggering alerts...
Sending alert state '1' to ThingSpeak...
Alert Uploaded! Server Response Entry ID: 6
Local alerts silenced.
Cooling down for 15 seconds to obey cloud server rules...
Area clear. Sending reset state '0' to ThingSpeak...
Reset Uploaded! Server Response Entry ID: 7

System reset complete. Scanning for movement...
```

9. Dashboard results.



9.0 Coding

```
import RPi.GPIO as GPIO
import time
import requests  # Handles the HTTP connection to ThingSpeak

# Use BCM numbering for Raspberry Pi GPIO pins
GPIO.setmode(GPIO.BCM)

# Pin Setup
PIR_PIN = 4
LED_PIN = 17
BUZZER_PIN = 27

# ------------------------------------------------
# THINGSPEAK CLOUD CONFIGURATION
# ------------------------------------------------
#  Make sure to paste your actual Write API Key between the quotes!
API_KEY = "756GZGCM5LH7XSMS"
URL = "https://api.thingspeak.com/update"

# Configure hardware pin directions
GPIO.setup(PIR_PIN, GPIO.IN)
GPIO.setup(LED_PIN, GPIO.OUT)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

# Force actuators off initially to guarantee a clean state
GPIO.output(LED_PIN, False)
GPIO.output(BUZZER_PIN, False)

print("=================================================")
print("     SDT 2133 IOT SECURITY SYSTEM INITIALIZING    ")
print("=================================================")
print("Warming up PIR sensor... Please wait 20 seconds.")
time.sleep(20)  # Crucial calibration window to prevent false startup alarms
print("System Live! Monitoring for intruder movements...\n")

try:
    while True:
        # Check if the hardware sensor pin reads HIGH (Movement found)
        if GPIO.input(PIR_PIN) == GPIO.HIGH:
            print("INTRUDER DETECTED! Triggering alerts...")

            # 1. Turn both local hardware actuators ON
            GPIO.output(LED_PIN, True)
            GPIO.output(BUZZER_PIN, True)

            # 2. Transmit '1' to Cloud (Spikes line chart UP, triggers Red Lamp)
            print("Sending alert state '1' to ThingSpeak...")
            try:
                payload = {'api_key': API_KEY, 'field1': 1}
```

```
            response = requests.get(URL, params=payload, timeout=5)
            print(f"Alert Uploaded! Server Response Entry ID: {response.text}")
        except Exception as e:
            print(f" Cloud Connection Error: {e}")

        # 3. Maintain local hardware alarm for 2 seconds
        time.sleep(2)

        # 4. Turn both local hardware actuators OFF
        GPIO.output(LED_PIN, False)
        GPIO.output(BUZZER_PIN, False)
        print("Local alerts silenced.")

        # 5. Rate Limit Protection Cooldown Buffer
        # ThingSpeak drops any data sent less than 15 seconds apart.
        print("Cooling down for 15 seconds to obey cloud server rules...")
        time.sleep(15)

        # 6. Transmit '0' to Cloud (Pulls line chart DOWN, turns Lamp off)
        print("Area clear. Sending reset state '0' to ThingSpeak...")
        try:
            payload = {'api_key': API_KEY, 'field1': 0}
            response = requests.get(URL, params=payload, timeout=5)
            print(f" Reset Uploaded! Server Response Entry ID: {response.text}")
        except Exception as e:
            print(f" Cloud Reset Failed: {e}")

        print("\n System reset complete. Scanning for movement...")

    # Idle state loop deceleration to keep Pi CPU utilization at ~0%
    time.sleep(0.1)

except KeyboardInterrupt:
    print("\n[INFO] Security system stopped manually.")

finally:
    # Safely releases the pins so they don't stay energized or throw warnings next run
    GPIO.cleanup()
    print("GPIO cleanup successful. Pins set to safe default states. System offline.")
```

10.0 Issues – *What difficulties have occurred*

During the development of this IoT Security System project, several challenges and technical issues were encountered. These issues helped improve our troubleshooting and problem-solving skills throughout the project development process.

10.1 PIR Sensor Sensitivity

The PIR sensor was sometimes too sensitive and detected unnecessary movements such as small object movements or environmental changes. This caused false alarms and unstable readings. To solve this issue, the sensor position and delay timing were adjusted carefully.

10.2 Buzzer Connection Problem

The buzzer occasionally failed to produce sound due to loose jumper wire connections and incorrect GPIO configuration. The issue was fixed by checking the wiring connections and ensuring the correct GPIO pins were used in the Python code.

10.3 Difficulty Connecting to ThingSpeak

There were several connection problems when sending data to the ThingSpeak cloud platform. Incorrect API keys, unstable internet connection, and delay limitations from ThingSpeak caused upload failures. The issue was solved by verifying the API key and adding a delay of 15 seconds between data transmissions.

10.4 Circuit Diagram and Diode Confusion

At the beginning of the project, there was confusion regarding the correct placement of resistors and LED polarity in the circuit diagram. Incorrect placement caused the LED not to function properly. Additional references and tutorials were used to understand the correct circuit configuration.

10.5 Difficulty Finding Raspberry Pi Simulator

It was difficult to find a simulator that fully supports Raspberry Pi 4 hardware and GPIO functions. Most available simulators only support Arduino or limited Raspberry Pi features. Therefore, the project was tested using actual hardware instead of simulation software.

11.0 Conclusion – *What I have learned, Future Recommendations*

In conclusion, this project successfully developed an IoT-based security monitoring system using Raspberry Pi 4, PIR sensor, LED, buzzer, and the ThingSpeak cloud platform. The system can detect human movement, activate warning alerts, and send real-time data to an online dashboard through an internet connection.

The project achieved its main objectives by enabling effective motion detection, visual and audio alerts, and remote monitoring through cloud integration. Throughout the development process, valuable knowledge and practical experience were gained in IoT technology, Raspberry Pi configuration, hardware integration, cloud communication, and Python programming.

This project shows that IoT technology can be used to create a simple, low-cost, and efficient security monitoring system suitable for homes, offices, and restricted areas. In the future, the system can be improved by adding a camera module, using multiple PIR sensors for wider coverage, and developing a mobile or web application for easier monitoring and control. Overall, the project provided meaningful exposure to IoT system development and real-time security monitoring technologies.

12.0 References

Raspberry Pi Foundation. (2025). *Raspberry Pi Documentation*. Retrieved from Raspberry Pi Documentation

MathWorks. (2025). *ThingSpeak Documentation*. Retrieved from ThingSpeak Documentation

Python Software Foundation. (2025). *Python Documentation*. Retrieved from Python Documentation

Random Nerd Tutorials. (2024). *Raspberry Pi PIR Motion Sensor Tutorial*. Retrieved from Random Nerd Tutorials PIR Guide

GPIO Zero Documentation Team. (2025). *GPIO Zero Documentation*. Retrieved from GPIO Zero Documentation

Electronics Hub. (2024). *PIR Sensor Working and Applications*. Retrieved from Electronics Hub PIR Sensor Article